Database After Graduation

Week 2 Assessment

Production Query Investigation & Optimization

Overview

This week's assessment simulates a real backend engineering task. Instead of writing basic queries, you will investigate performance, analyze execution plans, and think like a production engineer.

The goal is not just to "make the query work."
The goal is to understand how the database engine thinks, how indexes change behavior, and how workload impacts design decisions.

You will be working with the Week 2 dataset. Submission instruction is mentioned at the end of this document.

Step 0 – Database Setup

Before starting:

1. Go to Brightspace.
2. Navigate to: Week 2 Materials.
3. Download the file:

   `Create_db_Insert_to_db.sql`

4. Open MySQL Workbench.
5. Execute the entire script to create and populate the database.
6. Confirm the following tables exist:

- customers
- orders
- order_items
- products
- page_views

Optional but recommended:

```
ANALYZE TABLE orders;
ANALYZE TABLE order_items;
ANALYZE TABLE products;
ANALYZE TABLE customers;
```

This ensures optimizer statistics are up to date.

Part 1 – High-Value Customers (Performance Analysis)

Scenario

The business dashboard runs the following query every minute:

```
SELECT o.customer_id, COUNT(*) AS shipped_orders
FROM orders o
WHERE o.order_status = 'SHIPPED'
GROUP BY o.customer_id
ORDER BY shipped_orders DESC
LIMIT 25;
```

Tasks

1. Run EXPLAIN.
2. Identify:
    - access type
    - index used
    - estimated rows scanned
    - whether temporary or filesort is used
3. Explain why this query may degrade as the table grows.
4. Propose an index to improve performance.
5. Re-run EXPLAIN.
6. Compare before and after.
7. Explain which part improved:
    - filtering?
    - grouping?
    - sorting?
8. Estimate the write overhead introduced by your index.

Part 2 – Recent but Not Very Recent Customers

Scenario

The growth team wants customers who ordered in the last 14 days but not in the last 48 hours.

```
SELECT DISTINCT o.customer_id
FROM orders o
WHERE o.placed_at >= NOW() - INTERVAL 14 DAY
  AND o.customer_id NOT IN (
      SELECT o2.customer_id
      FROM orders o2
      WHERE o2.placed_at >= NOW() - INTERVAL 2 DAY
  );
```

Tasks

1. Explain why NOT IN can be problematic.
2. Rewrite the query using a safer production pattern.
3. Run EXPLAIN on both versions.
4. Compare estimated rows and access type.
5. Propose index improvements.
6. Explain which part of the query is likely the bottleneck.

Part 3 – Revenue by Product Category

Scenario

Analytics runs this report:

```
SELECT p.category, SUM(o.order_total) AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.order_status = 'PAID'
GROUP BY p.category;
```

Tasks

1. Run EXPLAIN.
2. Identify join order.
3. Identify which table is scanned first.
4. Identify potential row explosion.
5. Propose index improvements for:
    - filtering
    - join performance
6. Explain why join order matters.
7. Suggest one structural optimization beyond indexing.

Part 4 – Detecting Sorting Cost

Run:

```
EXPLAIN
SELECT *
FROM orders
WHERE order_status = 'PAID'
ORDER BY placed_at DESC
LIMIT 100;
```

Tasks

1. Does EXPLAIN show filesort?
2. What does Using temporary indicate?
3. Design a composite index to eliminate unnecessary sorting.
4. Re-run EXPLAIN and analyze changes.
5. Explain why column order inside the index matters.

Part 5 – Write-Heavy Scenario

Assume:

- orders table receives 8,000 inserts per minute.
- Reports run frequently.
- Storage cost is not constrained.

Answer in 400 words maximum:

1. Which indexes would you definitely keep?
2. Which ones would you reconsider?
3. How does workload shape index strategy?
4. What risks come from over-indexing?

Submission Instructions

Please submit a single PDF document that clearly answers all parts of the assessment.

Your submission must include:

• The original query (when required)
• Your rewritten or optimized query (if applicable)
• EXPLAIN output screenshots or copied results
• Clear explanations written in your own words
• Before and after comparisons when indexes are added
• Your reasoning behind each indexing decision

Important:

– Do not submit raw SQL files only
– Do not submit screenshots without explanation
– Explanations matter more than code

This is an engineering investigation, not a syntax test.

When discussing EXPLAIN results, clearly mention:

• access type
• key used
• estimated rows
• Extra column details (Using filesort, Using temporary, etc.)

For Part 5, keep your answer within 400 words. Focus on reasoning and tradeoffs.

How You Will Be Evaluated

You are not graded on whether you "guessed the perfect index."
You are graded on:

• Understanding of query execution
• Ability to interpret EXPLAIN
• Logical reasoning
• Awareness of tradeoffs
• Production thinking

If your reasoning is **sound**, even if your final index is not perfect, you will receive **strong credit**.

Expectations

This assignment is intentionally more realistic than typical academic SQL exercises. Some parts may feel challenging. That is expected.

Take your time. Run experiments. Add an index. Remove it. Compare results. Think about what changed and why.

If something surprises you in EXPLAIN, that is a good sign. It means you are learning how the optimizer behaves.

This week is about thinking like an engineer. You've got this.